

DD2352 Algorithms and Complexity
Spring 2026
Lecture 10: Decidability and Reductions

Per Austrin

2026-03-23

Agenda

Decision, Optimization, and Search

Decidability

Turing Reductions

Example Reductions

Complexity Intro Recap

We have now entered the “complexity” part of the course.

The field of **computational complexity** is devoted to understanding the *limitations* of computation.

Are there problems that *require* unreasonable amounts of resources to solve?

In previous lecture, we saw:

- ▶ Turing Machines and the RAM, two models of computation
- ▶ Comparison-based sorting requires $\Omega(n \log n)$ time

Agenda

Decision, Optimization, and Search

Decidability

Turing Reductions

Example Reductions

Algorithms, programs, and machines

This lecture we make no distinction between **algorithms**, **programs**, or **(Turing) machines**.

If we are being pedantic they are not the same thing.

But the **Church-Turing thesis** says that the concepts are equivalent!

If there is an algorithm there is a Turing machine, etc.

Decision, Optimization, and Search Problems

Three main types of problems:

- ▶ **Search problems:** answer is some “object” representing an actual solution to a problem (e.g. a maximum flow in a flow network)
“What is the optimal solution?”
- ▶ **Optimization problems:** the answer is a value, the maximum (or minimum) of some objective function over some search space.
“How good is the optimal solution?”
- ▶ **Decision problems:** the answer is just a single bit of information, “yes” or “no”.
“Does there exist a solution with value at least k ?”

In complexity part of the course, we will focus on **decision problems**.

In general (for the problems we study in this course) we do not lose much by this: if we can solve one of them we can solve all of them.
(See exercise session this week!)

Problem Instances

In a general problem, we often think of the input to an algorithm as having some structure, e.g.:

- ▶ a graph
- ▶ some numbers
- ▶ a string of letters

But these will always be stored in the computer memory as a sequence of bits.

So at the lowest level, we can always think of the input to an algorithm as a binary string $x \in \{0, 1\}^*$.

Agenda

Decision, Optimization, and Search

Decidability

Turing Reductions

Example Reductions

Decidability

A decision problem is **decidable** if there exists an algorithm for solving the problem.

Note: this is without any efficiency considerations!

We only care that the algorithm eventually terminates, but it is allowed to take a loooooooooooooooooooooooooooooooooong time

Is this even a meaningful notion?

Surely all problems can be solved if we just throw enough computer time on them?

The Halting Problem

HALTING

Input: Turing machine M , and input x to M .

Output: Does M halt when run on the input x ?

Theorem

*The HALTING Problem is **undecidable**.*

Undecidability of Halting (1/2)

Suppose for contradiction that there exists an algorithm `Halts` that decides HALTING.

Concretely, for any Turing machine M and input x ,

$$\text{Halts}(M, x) = \begin{cases} \text{true} & \text{if running } M(x) \text{ halts} \\ \text{false} & \text{if } M(x) \text{ goes into infinite loop} \end{cases}$$

Let us construct a new algorithm A that attempts to “trick” `Halts` into failing.

A works as follows:

```
1 function  $A(M)$ :  
2   if Halts( $M$ ,  $M$ ) then enter an infinite loop  
3   return
```

Undecidability of Halting (2/2)

What happens if we run $\text{Halts}(A, A)$?

There are two possible cases:

1. $\text{Halts}(A, A)$ returns true.

In other words, the Halts algorithm claims $A(A)$ halts. But for $A(A)$ to actually halt, $\text{Halts}(A, A)$ would need to return false.

Contradiction!

2. $\text{Halts}(A, A)$ returns false.

In other words, the Halts algorithm claims $A(A)$ goes into an infinite loop. But this only happens if (i) $\text{Halts}(A, A)$ returns true, or (ii) $\text{Halts}(A, A)$ itself goes into an infinite loop.

Contradiction!

In both cases, we reach a contradiction. We conclude that such a machine Halts cannot exist.

Variants of the Halting Problem

HALTING ON EMPTY INPUT

Input: Turing machine M .

Output: Does M halt when run on no input?

HALTING ON ALL INPUTS

Input: Turing machine M .

Output: Does M halt on *all* possible inputs?

Using the undecidability of the HALTING problem, we can prove that both of these are also undecidable.

Other examples of undecidable problems

DIOPHANTINE EQUATION

Input: Polynomial $P(x_1, \dots, x_n)$ with integer coefficients.

Output: Is there a positive integer solution to $P(x_1, \dots, x_n) = 0$?

Examples:

- ▶ Does $x^2 + y^2 = z^2$ have a positive integer solution?

Answer: **YES**.

(Pythagorean triples, e.g. $x = 3$, $y = 4$, $z = 5$)

- ▶ Does $x^3 + y^3 = z^3$ have a positive integer solution?

Answer: **NO**.

(Not obvious! Fermat's Last Theorem.)

- ▶ Does $x^2 - 313y^2 = 1$ have a positive integer solution?

Answer: **YES**.

(Not obvious! Smallest solution is $x = 32188120829134849$,
 $y = 1819380158564160$)

Other examples of undecidable problems

PLANE TILING

Input: A set of types of square-shaped tiles.

Output: Is it possible to tile the entire plane $\mathbb{R}^2$ with tiles of the given types so that the pattern matches everywhere?

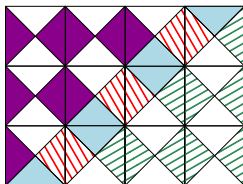
Example:

Can these tiles be used to tile the plane:



Answer: **YES**

Start of a tiling (hopefully obvious how it extends in all directions):



Other examples of undecidable problems

POST CORRESPONDENCE PROBLEM

Input: List of word pairs $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Output: Is there a finite sequence of indices $i_1, i_2, \dots$ så att
 $x_{i_1}x_{i_2}x_{i_3}\dots = y_{i_1}y_{i_2}y_{i_3}\dots$

Example 1:

- ▶ $(abb, bbab), (a, aa), (bab, ab), (bab, ab), (baba, aa), (aba, a)$
- ▶ Correspondence achieved with 2, 1, 1, 4, 1, 5:

a	a	b	b	a	b	b	b	a	b	a	a	b	b	a	a
2	1		1		4		1		5						
2		1		1		4		1		5					

Example 2:

- ▶ $(bb, bab), (a, aa), (bab, ab), (bab, ab), (baba, aa), (aba, a)$
- ▶ No correspondence possible.

Recursively Enumerable Problems

Many of these problems have a property in common:
there exists an “algorithm” which terminates whenever the answer is “yes” but does not terminate on “no” instances

- ▶ In HALTING, simply run the Turing machine. If it terminates, return “yes”, otherwise keep going.
- ▶ In DIOPHANTINE EQUATIONS, enumerate all integer solutions and try them.

If the answer is “yes” then eventually we find the solution.

Problems which have algorithms like this are called **recursively enumerable**.

Agenda

Decision, Optimization, and Search

Decidability

Turing Reductions

Example Reductions

Reductions

The most successful and important technique in computational complexity is **reductions** – relating the difficulty of different problems to each other even though we might not know exactly how hard they are.

Turing Reductions

A **Turing reduction** (also called **Cook reduction**) from a problem X to a problem Y is an efficient algorithm for problem X which is allowed to assume an algorithm for Y as a black box.

```
1 function SolveX(A):  
2   Do some stuff  
3   Build some instance  $B$  of problem  $Y$   
4   Call SolveY( $B$ ) and use the answer somehow  
5   Do other stuff  
6   Call SolveY( $B'$ ) on some other instance  $B'$   
7   Do more stuff  
8   ...  
9   ???  
10  ...  
11  Figure out answer to the input  $A$ 
```

Notation: we write $X \leq_T Y$ to denote this (“ X Turing-reduces to Y ”)

Intuition: X is not (significantly) harder than Y

Positive Uses of Reductions

Reductions between problems can be used as a positive tool and as a way of finding new algorithms.

In fact we have already done this several times in the course.

- ▶ When looking at applications of Max-Flow, we reduced various problems to MaxFlow

$$\text{MINEDGE CUT} \leq_T \text{MAXFLOW}$$

$$\text{MAXFLOWWITHVERTEXCAPACITIES} \leq_T \text{MAXFLOW}$$

$$\text{BIPARTITEMATCHING} \leq_T \text{MAXFLOW}$$

and several others...

- ▶ In Lecture 5, the way we constructed solutions for the WEIGHTED INTERVAL SCHEDULING problem given our dynamic programming algorithm was really a reduction.

Reductions allow us to think about problems in a “modular” way. We do not need to know *how* to solve MAXFLOW to establish these relations.

Negative Uses of Reductions

In the complexity part of the course, we will primarily focus on the flip side of this:

If $X \leq_T Y$ and X is “hard”, then Y must be “hard” as well.

If we have one hard problem X , we can show that other problems are hard by reducing *from* X to other problems.

Example: Consider the following two facts.

1. HALTING is an undecidable problem, and
2. $\text{HALTING} \leq_T \text{HALTINGONEMPTYINPUT}$

We conclude that HALTINGONEMPTYINPUT is also undecidable.

Friendly warning: it is very easy to get the direction of the reductions wrong!

Agenda

Decision, Optimization, and Search

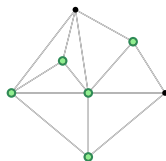
Decidability

Turing Reductions

Example Reductions

Reduction Example 1: Vertex Cover and Independent Set

A vertex cover in a graph G is a set of vertices that “touches” every edge of G .

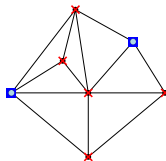


VERTEX COVER

Input: Graph G and parameter k

Output: Does G have a vertex cover of size $\leq k$?

An independent set in a graph G is a set of vertices with no edges between them.



INDEPENDENT SET

Input: Graph G and parameter k

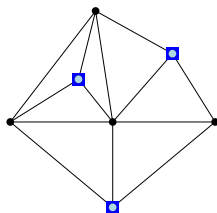
Output: Does G have an independent set of size $\geq k$?

Vertex Cover and Independent Set, continued

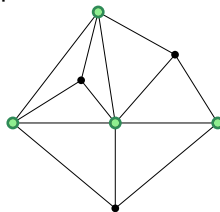
Claim

S is an independent set in G if and only if $V - S$ is a vertex cover.

Proof by picture:



A large independent set



A small vertex cover

Reduction establishing $\text{INDEPENDENT SET} \leq_T \text{VERTEX COVER}$:

```
1 function IndependentSet( $G, k$ ):  
2   return VertexCover( $G, |V| - k$ )
```

(We could also show $\text{VERTEX COVER} \leq_T \text{INDEPENDENT SET}$)

Example 2: Set Cover

SET COVER

Input: A collection of sets $S_1, \dots, S_m \subseteq U$ over some universe U , and parameter k

Output: Does there exist $\leq k$ of the sets whose union equals U ?

Example:

$$U = \{1, 2, 3, 4, 5\}$$

$$S_1 = \{1, 4, 5\}$$

$$S_2 = \{1, 2, 4\}$$

$$S_3 = \{3, 4\}$$

$$S_4 = \{2, 5\}$$

$$S_5 = \{3, 4, 5\}$$

$$k = 2$$

The answer is “yes”, because $S_2 \cup S_5 = \{1, 2, 3, 4, 5\} = U$

Reducing Vertex Cover to Set Cover

We can think of the VERTEX COVER problem as a type of set covering problem:

- ▶ The universe U that we are trying to cover are the edges of the graph
- ▶ For each vertex u of the graph, there is a set S_u of edges that are covered if we pick that vertex (the edges that touch u)

Reduction establishing VERTEX COVER $\leq_T$ SET COVER:

```
1 function VertexCover( $G, k$ ):  
2   Let  $U = E$   
3   For each vertex  $u \in V$ , construct a set  $S_u$  consisting of all  
   edges touching  $u$   
4   return SetCover( $\{S_u\}, U, k$ )
```

(Unlike in the previous example, this only goes one direction and does not establish that SET COVER $\leq_T$ VERTEX COVER.)

Example 3: 3-Sat

A 3-CNF formula is a logic formula of the form:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee \neg x_2 \vee \neg x_{17}) \wedge \dots$$

In words:

- ▶ A **conjunction** (logical AND) of a set of **clauses**
- ▶ Each clause is a **disjunction** (logical OR) of exactly 3 **literals**
- ▶ Each literal is either a variable or a negated variable.

3-SAT

Input: A 3-CNF formula Φ .

Output: Is Φ satisfiable?

Another perspective on 3-Sat

$$(x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee \neg x_2 \vee \neg x_{17}) \wedge \dots$$

An alternative way of thinking about 3-Sat:

- ▶ in a satisfying assignment, there is (at least) one literal in each clause that is true
- ▶ we can try thinking of the problem as being about selecting one “representative” literal for each clause, to satisfy it
- ▶ what constraints do we have on this selection?
- ▶ if we select x to be a true literal in one clause, then we cannot choose $\neg x$ to be a true literal in any other clause.
- ▶ this type of conflict is similar to the independent set problem.

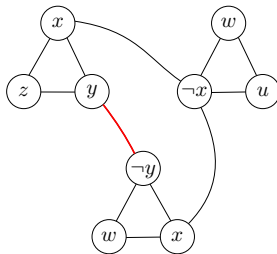
Reducing 3-Sat to Independent Set

Reduction:

```
1 function 3Sat( $\Phi$  on  $n$  variables and  $m$  clauses):  
2   Create graph  $G$  with one vertex per variable appearance, and  
   add edges between (i) all variable appearances from the  
   same clause, and (ii) all appearances of  $x$  and  $\neg x$  for each  
   variable  $x$ .  
3   return IndependentSet( $G, m$ )
```

Example of reduction:

$$\begin{aligned}\Phi = & (x \vee \textcolor{red}{y} \vee z) \\ & (w \vee u \vee \neg x) \\ & (\neg \textcolor{red}{y} \vee x \vee w)\end{aligned} \implies$$



Reducing 3-Sat to Independent Set

Reduction:

```
1 function 3Sat( $\Phi$  on  $n$  variables and  $m$  clauses):  
2   Create graph  $G$  with one vertex per variable appearance, and  
   add edges between (i) all variable appearances from the  
   same clause, and (ii) all appearances of  $x$  and  $\neg x$  for each  
   variable  $x$ .  
3   return IndependentSet( $G, m$ )
```

To show that this “works”, we need to prove:

Lemma

The 3-SAT instance Φ is satisfiable if and only if the graph G has an independent set of size $\geq m$.

(See book §8.2 for proof.)

Summary of Examples

From the examples we have seen, we can conclude:

$$3\text{-SAT} \leq_T \text{INDEPENDENT SET} \leq_T \text{VERTEX COVER} \leq_T \text{SET COVER}$$

It is unknown whether efficient algorithms exist for these problems, but we have shown that if an algorithm exists for any one of them, then an algorithm exists for 3-SAT.

And on the other hand if 3-SAT does not have an efficient algorithm, then none of these problems do.

Karp Reductions

The reductions we saw today were all like this:

```
1 function SolveX(instance A of problem X):  
2   |   Create an instance B of problem Y in some way from A  
3   |   return SolveY(B)
```

A reductions of this form is called a **Karp Reduction** (or a *polynomial-time many-one reduction*).

Notation: we write $X \leq_p Y$ to denote this (“*X* Karp-reduces to *Y*”)

In words: $X \leq_p Y$ if given any instance *A* of problem *X*, we can in polynomial time construct an instance $f(A)$ of problem *Y* such that the answer to *A* is the same as the answer to $f(A)$.

Caveat: *the book never introduces Karp reductions and only defines Turing reductions. This leads to subtle but crucial differences in the material that follows. We will follow the standard definitions and use Karp reductions.*

Lab 2

$X \leq_p Y$ if there is a reduction of this form:

```
1 function SolveX(instance A of problem X):  
2   Create an instance B of problem Y in some way from A  
3   return SolveY(B)
```

Lab 2 in the course is about creating a reduction like this.

In particular your job is to implement “line 2” above.

- ▶ *Theory part:*
you should be ready to start this week.
- ▶ *Programming part:*
you should be ready to start after lecture 12.

(Warning: you will probably initially find it confusing what exactly X and Y should be in the lab. But figuring out what X and Y are, and *why* they are what they are, is an important part of understanding the lab.)

Upcoming Week In The Course

- ▶ Tomorrow 15-17: Lecture 11
P, NP, and NP-completeness
- ▶ Thu 15-17: Exercise 7
Undecidability, reductions, and decision vs optimization vs search
- ▶ Fri 8-10: Lab
Lab 2 help
Also possible to present (both Lab 1 and Lab 2)